

CASO PRÁCTICO · PRODUCT ENGINEER

Planificar un torneo de hockey sin que el calendario sea el problema

Cómo abordé un problema real de coordinación y lo convertí en una solución funcional

Francisco Pieper Caussade

Julio 2026

Juego hockey césped hace años

Club Sport Francés • Varones Adulto Primera División • Torneo FEHOCH

1

Semestral

Cada campeonato dura un semestre completo

2

Ventana rígida

Solo sábado y domingo, de 08:00 a 19:00

3

Recurso escaso

Pocas canchas para muchas categorías

4

Hecho a mano

Una persona lo arma en una planilla

Elegí este problema porque lo vivo: sé dónde duele y por qué las soluciones obvias fallan.

Tres cosas que pasaron de verdad



Partidos en fechas imposibles

Se agendaron partidos el Día de la Madre y a la hora de la final del Mundial. Resultado: la gente no llega.



Recuperaciones nocturnas

Un equipo pide cambiar su horario y el partido termina un martes por la noche. Mucha gente no puede por trabajo.



“Hora a definir”

Las fechas lejanas quedan sin hora asignada. Nadie puede organizarse con anticipación.

No son hipótesis. Son incidentes que motivaron cada restricción del sistema.

Cuatro actores que quieren cosas distintas



Equipos

Buenos horarios.
No jugar a las 8 am
siempre los mismos.



Árbitros

Dos por partido.
Sin choques y sin
huecos muertos.



Clubes

Agrupar a todos sus
equipos el mismo día
y en el mismo recinto.



Federación

Llenar las canchas y
cumplir compromisos
de televisión.

El conflicto entre estos objetivos es el problema. Si todos quisieran lo mismo, sería una planilla.

Primero entender el dominio. Después escribir código.

Antes de tocar el editor busqué el reglamento oficial de la Federación Chilena de Hockey y modelé el problema con datos reales.

1

Reglamento FEHOCH 2026

Duraciones, categorías, equipos y reglas operativas reales

2

Supuestos corregidos

Tres cosas que yo creía y estaban mal

3

Spec escrito antes del código

Un plan que define el alcance y lo que NO se construye

Lo que cambió al leer el reglamento

- La duración NO es única: varía por categoría
- La localía manda: el partido va al recinto del equipo local
- Viña tiene UNA cancha para la localía de tres clubes

La capacidad total alcanza. Los cuellos de botella, no.

~1.000

partidos en un
semestre

4

canchas para varones
en todo Santiago

1

cancha en Viña para
la localía de 3 clubes

En agregado sobra capacidad. El problema son los cuellos de botella locales — y ahí es donde el criterio decide.

Un asistente que arma el semestre completo

Arma el calendario del semestre y —lo importante— lo mantiene válido cada vez que algo cambia.

ENTRA

- Enfrentamientos del semestre
- Canchas y su regla de acceso
- Disponibilidad de árbitros
- Feriados y eventos masivos
- Preferencias en texto libre



SALE

Calendario completo

Cada partido con cancha, día y hora concretas

Justificación

Por qué cada partido quedó donde quedó

Alertas

Qué no se pudo satisfacer y por qué

Qué decide el sistema



Restricciones duras

Se cumplen al 100%. Son invariantes: hay tests que lo comprueban.

- El partido va a la cancha de uno de los dos equipos
- Cada equipo, local en la mitad de sus partidos
- Ningún equipo en dos partidos simultáneos
- Dos árbitros por partido, sin choques
- Nada en feriados ni eventos masivos
- Nada de recuperaciones entre semana



Restricciones blandas

No se cumplen siempre. Se maximizan con una función de puntaje.

- Horas fáciles de recordar: 13:00, 13:30
- Equidad: repartir los horarios malos
- Preferencia por sábado, alternando géneros
- Categorías menores en las primeras horas
- Agrupar los equipos de un mismo club
- Minimizar huecos muertos de cancha

No dejo que el modelo de lenguaje arme el calendario

Un LLM no garantiza restricciones duras: puede poner dos partidos en la misma cancha a la misma hora y sonar convincente al explicarlo. Así que separé responsabilidades.

Solver determinístico

Asigna los partidos.
Garantiza factibilidad.
Es testeable y reproducible.

Lo que exige determinismo

Agente LLM

Interpreta mensajes de WhatsApp.
Explica por qué un equipo quedó a una hora determinada.

Lo que exige lenguaje

El LLM es la superficie. El solver es la garantía. Usar IA con criterio es saber dónde NO usarla.

Planificar es lo fácil. Replanificar es lo difícil.

El calendario inicial se arma una vez, sin apuro. Pero después empieza a moverse: un equipo pide cambio, llueve, un árbitro se cae. Cada cambio puede romper todo lo demás.

Lo que preguntan

“¿Podemos mover el partido del sábado?”
“¿Cuándo juega mi equipo?”
“¿Por qué quedamos a las 8 am?”

El LLM entiende la pregunta

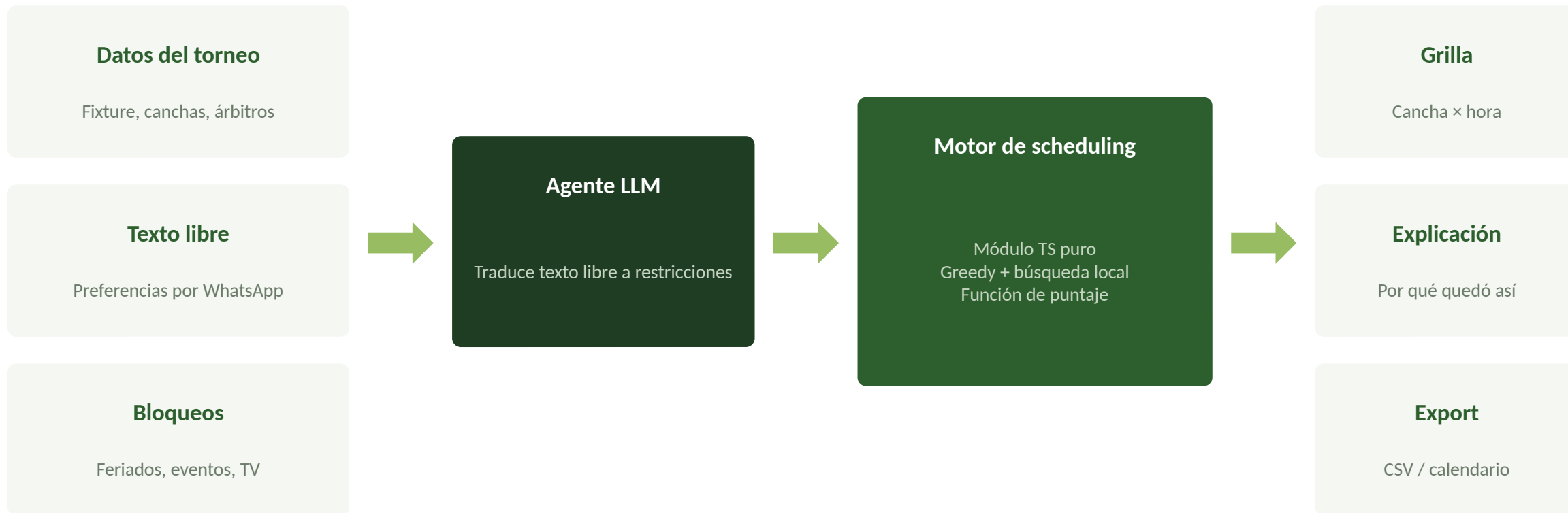
Lo que hay que responder

Qué alternativas son legales, cuál es la mejor,
y qué se rompe si se elige cada una.

El solver garantiza la respuesta

El valor no está en armar el calendario. Está en mantenerlo válido cuando la realidad lo golpea.

Cómo funciona



El motor no sabe que existe la web: es un módulo TypeScript puro, sin dependencias de framework. Se puede testear aislado y mover a otro runtime sin tocarlo.

Qué tiene que pasar para decir que sirve

100%

de partidos con hora concreta

Cero “hora a definir”

0

violaciones de restricciones duras

Verificado con tests

Horas → segundos

tiempo de armado

El cambio más tangible

Menor varianza

en horarios malos por equipo

Equidad medible

El baseline existe para mostrar el contraste

Construí primero un scheduler deliberadamente tonto que solo evita chocar canchas. Sirve como punto de comparación honesto.



Sin optimizar

- 24 choques de equipo
- Un equipo citado 3 veces a la vez
- Los 28 partidos apretados el sábado
- Domingo completamente vacío

Técnicamente válido. Humanamente inservible.



Con el solver

- Cero choques de equipo y de árbitro
- Horarios repartidos con equidad
- Sábado y domingo balanceados
- Cada decisión explicable

La demo del jueves compara ambos sobre los mismos datos.

Dirigí al agente. No le delegué el criterio.

1 Spec antes que código

Escribí el plan completo —alcance, restricciones, arquitectura— antes de generar una línea.

2 Non-goals explícitos

Cada instrucción al agente dice también qué NO construir. Sin eso, se dispersa.

3 QA sobre lo que entregó

Audité su trabajo ejecutando el código. Reportó “cero choques de cancha”: cierto, pero omitió 24 choques de equipo.

4 Tests como invariantes

Las restricciones duras no se confían: se comprueban.

La velocidad la da el agente. El criterio de qué construir y qué verificar, no.

Qué no construí, y por qué

Un solo deploy, no microservicios

El problema cabe en memoria: ~1.000 partidos se resuelven en menos de un segundo. Separar servicios sería complejidad sin problema que la justifique.

Motor propio, no una librería de solver

Poder explicar cómo decide vale más que un punto de optimalidad. Una caja negra no se puede defender.

Sin ascensos, descensos ni playoffs

Son reglas de competencia, no de calendarización. Están fuera del problema que elegí resolver.

Dónde sí cambiaría de arquitectura: cuando la optimización del semestre completo pase de los 10 segundos, el solver sale del request y pasa a ser un job encolado.

**Un problema de coordinación
que se resuelve con criterio,
no con más planillas.**

El problema

Vivido, no inventado

El método

Datos reales antes que código

La solución

Solver determinístico + IA donde
corresponde

Gracias.